🦷 Resumen Definitivo para la Defensa — Clínica Dental Ros

> IMPORTANT
>
> Este resumen recorre **las 8 capas de la solución** de abajo hacia arriba, explicando **qué hace cada capa** y **qué hacen sus archivos más importantes**.

🧬 1. Capa `Domain.Model` (Entidades del Negocio)

- **¿Qué hace la capa?** Representa los objetos del mundo real de la clínica. **No depende de ninguna otra capa ni de Entity Framework**. Usa el patrón de **Dominio Rico** (setters privados y validaciones en constructores/métodos para que una entidad nunca pueda existir con datos inválidos).

Archivos principales:

- : **Clase abstracta (`abstract`)** base. No se puede instanciar con `new`. Agrupa los datos comunes (`Nombre`, `Apellido`, `Dni`, `Telefono`, `Mail`, `Domicilio`) y sus validaciones (`Validate()`, `TelefonoValido()`).
- **y** : Clases hijas que heredan de `Persona`. `Paciente` agrega `ObraSocial`, `NumeroAfiliado` y `EstadoHabilitado`. `Odontologo` agrega `NumMatricula` y `Especialidad`.
- : Representa la **reserva de horario en la agenda**. Controla la **máquina de estados** del turno (`Pendiente` ➔ `Presente` ➔ `Atendido`, o `Cancelado` / `NoAsistido`) con métodos como `ConfirmarPresencia()`, `Atender()`, `Cancelar(motivo)` y `RegistrarAusencia()`.
- : Representa el **acto médico real en el consultorio** cuando el turno fue atendido (`Diagnostico`, `Tratamiento`, `Observaciones`, `AnestesiaLocal`, `Radiografias` y calificación con estrellas).
- **+** : Maneja el cobro de la atención. Calcula automáticamente el subtotal (honorarios + insumos), el descuento según el porcentaje de cobertura de la Obra Social y el monto final a pagar por el paciente.
- **y** : Objetos que agrupan filtros opcionales de búsqueda (Patrón Criteria).

🗄️ 2. Capa `Data` (Acceso a Datos / Persistencia)

- **¿Qué hace la capa?** Es la **ÚNICA capa que se comunica con SQL Server**. Implementa el **Patrón Repository** usando **Entity Framework Core 8** (y ADO.NET puntual) para que el resto del sistema no sepa qué motor de base de datos se usa.

Archivos principales:

- : Es la sesión y conexión con la base de datos (hereda de `DbContext`).
  - Define los `DbSet<T>` (las tablas de la BD).
  - En `OnModelCreating` usa **Fluent API** para configurar claves, precisión decimal `(18,2)` para dinero, y reglas de borrado: `DeleteBehavior.Restrict` (bloquea borrar pacientes/odontólogos que tengan turnos), `Cascade` (si se borra una factura, se borran sus ítems) y `SetNull` (si se borra una obra social, el paciente queda con `ObraSocialId = null`).
  - En `SeedDatabase` carga datos iniciales de prueba con `.HasData(...)`.
- : Maneja **Transacciones ACID ("todo o nada")** con `EjecutarEnTransaccionAsync`. Envuelve las operaciones en `CreateExecutionStrategy()` (para soportar reintentos de conexión de SQL Server), abre `BeginTransactionAsync()`, hace `CommitAsync()` si todo sale bien, o `RollbackAsync()` + `ChangeTracker.Clear()` si algo falla.
- **y** : Ejecutan las consultas a la BD usando LINQ (`.Include()`, `.ThenInclude()`, `.Where()`, `.FirstOrDefaultAsync()`, `.ToListAsync()`) y ADO.NET (`SqlConnection` / `SqlDataReader` en el `GetByCriteriaAsync` de Pacientes).

📦 3. Capa `DTOs` (Objetos de Transferencia de Datos)

- **¿Qué hace la capa?** Contiene clases simples (solo propiedades `get; set;`, sin lógica) que viajan convertidas en JSON entre la `WebAPI` y los clientes (`WindowsForms` y `Blazor`).
- **¿Para qué sirve?** Para no exponer las entidades del dominio ni datos sensibles (como `PasswordHash`), evitar referencias circulares al serializar a JSON, y enviar datos ya "aplanados" (como `PacienteNombre` u `OdontologoNombre`) listos para mostrar en las grillas.

🧠 4. Capa `Application.Services` (Lógica de Negocio)

- **¿Qué hace la capa?** Es el **cerebro del sistema**. Coordina los repositorios, aplica las reglas de negocio de la clínica, ejecuta las transacciones y convierte Entidades a DTOs mediante `MapToDTO`.

Archivos principales:

- :
  - `LoginAsync`: Busca al usuario, verifica su contraseña con `PasswordHasher.Verify` y devuelve el token.
  - `GenerateTokenResponse`: Fabrica el **Token JWT** metiendo adentro los **Claims** (`Id`, `Username`, `Rol`, `EntidadId`) y lo firma digitalmente con `HMAC-SHA256` usando la `SecretKey`.
  - `RefreshTokenAsync`: Renueva tokens expirados hace menos de 24 hs sin volver a pedir contraseña.
- : Clase **estática (`static`)** que hashea contraseñas (`Hash`) generando un **Salt aleatorio de 16 bytes** con `HMACSHA256` (formato `"salt:hash"` en Base64) y las verifica (`Verify`) usando `FixedTimeEquals`.
- :
  - `ReservarTurnoAsync`: Valida que la fecha sea futura, que el paciente esté habilitado, que el odontólogo no tenga otro turno a esa hora y que el paciente no tenga dos turnos el mismo día.
  - `RegistrarAusenciaAsync`: Usa `TransactionManager` para hacer 3 pasos atómicos: marca el turno `NoAsistido`, crea una `Multa` y pasa al paciente a `Inhabilitado`.
- + :
  - `RegistrarConsultaAsync`: En una transacción marca el turno como `Atendido`, guarda la `Consulta`, **descuenta el stock** de los insumos usados y **crea automáticamente la `Factura`** aplicando el descuento de la Obra Social.
- : Valida DNI y Email únicos; al crear un paciente le crea automáticamente su `Usuario` web en una transacción; y al eliminar verifica con `GetRelacionesAsync` si tiene turnos, facturas o multas antes de borrar.

⚙️ 5. Capa `WebAPI` (Servidor HTTP REST)

- **¿Qué hace la capa?** Expone el backend a través de rutas HTTP (Endpoints usando **Minimal APIs** de ASP.NET Core 8) para que los frontends puedan consumirlo.

Archivos principales:

- *(llamado desde `Program.cs`)*: Configura e inicia todo el servidor:
  1. Registra `TurnoMolarDbContext` con SQL Server y `EnableRetryOnFailure`.

2. Configura CORS (`AllowAll`).
3. Configura la validación de Tokens JWT con la `SecretKey` .
4. Define las políticas de roles (`AdminOnly` , `AdminOrRecepcionista` , `AdminOrOdontologo` , `StaffOnly`).
5. Registra todos los Repositorios y Servicios con **Inyección de Dependencias (`AddScoped`)**.
6. Ejecuta `Database.EnsureCreated()` al arrancar y mapea todos los Endpoints.

- : Manejador global de excepciones (`IExceptionHandler`). Atrapa errores de todas las capas y los traduce a respuestas HTTP JSON:
  - `ArgumentException` / `InvalidOperationException` ➔ **400 Bad Request** (con el mensaje de validación).
  - `DbUpdateException` ➔ **409 Conflict** (registro en uso por otras tablas).
  - Cualquier otro error ➔ **500 Internal Server Error** (y lo anota en consola con `ILogger`).
- : Lee los claims del token del usuario (`EsPersonal()` , `PacienteId()` , `PuedeAccederAPaciente()`) para asegurar que un paciente solo pueda ver sus propios turnos, facturas y consultas (si intenta ver los de otro, devuelve `403 Forbid`).
- **y** : Definen las rutas HTTP (`MapGet` , `MapPost` , `MapPut` , `MapDelete`), validan permisos y llaman al servicio correspondiente.

📡 6. Capa `API.Clients` (Clientes HTTP Compartidos)

- **¿Qué hace la capa?** Es usada tanto por `WindowsForms` como por `Blazor.Server` para hacer las peticiones HTTP a la `WebAPI` sin duplicar código.

Archivos principales:

- : Interfaz que define cómo obtener los datos de la sesión activa (`GetToken()` , `GetRol()` , `IsAuthenticated()`).
- : Clase **abstracta** padre de todos los clientes:
  - **`GetConfiguredClientAsync()`** : Antes de salir hacia la API, agarra el `HttpClient` y le pega el token JWT en el header `Authorization: Bearer <token>` .
  - **`VerificarRespuestaAsync()`** : Al volver de la API, revisa el código HTTP: si es `200 OK` sigue; si es `401` avisa sesión expirada; si es `400` / `409` extrae el `"message"` del JSON y lanza excepción para mostrarlo en pantalla.
  - **`GetAsync` , `PostAsync` (2 sobrecargas), `PutAsync` , `DeleteAsync`** : Métodos protegidos que convierten entre C# y JSON (`PostAsJsonAsync` , `ReadFromJsonAsync`).

- `*ApiClient.cs` (`TurnoApiClient`, `PacienteApiClient`, `AuthApiClient`, **etc.)**: Clases hijas que heredan de `BaseApiClient` y llaman a cada ruta específica (ej: `"/api/turnos/reservar"`).

🔐 7. Capas `API.Auth.WindowsForms` y `API.Auth.Blazor.Server`

- **¿Por qué están separadas?** Porque Escritorio y Web guardan la sesión del usuario en lugares distintos:
  - : Guarda el único token de la PC en variables en memoria RAM.
  - : Se registra con `AddScoped` (una instancia separada por cada usuario conectado) y guarda la sesión en el `ProtectedSessionStorage` del navegador para que no se borre al tocar F5.
  - **y** : Integran el rol del usuario con el sistema de autorización visual de Blazor (`[Authorize]`).

🖥️🌐 8. Los Frontends: `WindowsForms` y `Blazor.Server`

- **¿Qué hacen?** Son las interfaces de usuario (Escritorio y Web). **Nunca tocan la base de datos ni `Application.Services` directamente**; todo lo hacen a través de `API.Clients`.

Archivos principales en `WindowsForms`:

- : Configura el manejador global de errores visuales, inicializa `WindowsFormsAuthService` y ejecuta el bucle `LoginForm` ➔ `HomeForm`.
- : Pide usuario y clave, llama a `AuthApiClient.LoginAsync` y guarda la sesión en `WindowsFormsAuthService`.
- : Ventana contenedora **MDI (`IsMdiContainer = true`)**. Arma el `MenuStrip` habilitando o deshabilitando menús según el rol (`_authService.GetRol()`) y abre los `UserControl` (`*Control.cs`) en ventanas hijas MDI sin duplicarlas.

Archivos principales en `Blazor.Server`:

- : Registra con `AddScoped` la sesión (`BlazorServerAuthService`), el `HttpClient` y todos los `*ApiClient` por cada usuario conectado.
- `Shared/` **(y)**: Estructura visual general y menú lateral que muestra opciones según `Permisos.Puede(rol, ...)`.
- `Pages/*.razor` (`Login.razor`, `Turnos.razor`, `Pacientes.razor`, `AtencionClinica.razor`, `PortalPaciente.razor`, **etc.)**: Pantallas web divididas en 3 partes: Directivas (`@page`, `@inject`),

HTML con `@if` / `@foreach` / `@bind-Value`, y bloque `@code` con `OnInitializedAsync()`.

- **`Components/*.razor` (`*Modal.razor`)**: Ventanas emergentes reutilizables para altas, ediciones y confirmaciones.